

Late Breaking Results: An Efficient and Scalable Track Assignment with GPU Parallelism

Genggeng Liu¹, Pengcheng Huang¹, Zepeng Li¹, Wen-Hao Liu², Xing Huang^{3,4,*}, and Wenzhong Guo¹

¹College of Computer and Data Science, Fuzhou University, Fuzhou, China

²NVIDIA, Hsinchu, Taiwan

³School of Computer Science, Northwestern Polytechnical University, Xi'an, China

⁴Fuzhou University of International Studies and Trade, China

liu_genggeng@126.com; 231027160@fzu.edu.cn; li_zepeng@yeah.net; wenhliu@nvidia.com; xing.huang@nwpu.edu.cn; guowenzhong@fzu.edu.cn

Abstract—The track assignment has been introduced between global routing and detail routing. Based on the independence and divisibility of track assignment, we propose a GPU-accelerated parallel track assignment algorithm. To estimate routability more accurately, the proposed algorithm simultaneously considers global and local nets, and incorporates several strategies for optimization. Moreover, an asynchronous parallelism strategy is proposed to divide the computation of routing resources and the track assignment into fine-grained tasks. Experimental results show that, compared to related work, our algorithm achieves a significant speedup with a better routability estimation.

I. INTRODUCTION

Traditional VLSI routing is typically divided into two stages: global routing and detail routing [1]. The global routing stage is to create a rough routing guidance for the subsequent detail routing. Based on global routing results, detail routing generates accurate interconnections, and all nets are assigned to appropriate tracks according to design rules. With the increasing routing complexity and net density, global routing is getting more difficult to meet the requirements of detail routing. Therefore, the track assignment has been introduced as the initial stage of detail routing to reduce the mismatch between global routing and detail routing [2].

Track assignment can effectively reduce overlaps between wires and blocks to minimize the mismatch between global routing and detail routing by providing a better initial solution for detail routing. NTA [3] proposed a negotiation-based track assignment algorithm that extracts wires from the global routing results and uses a heuristic approach to assign all wires to tracks to minimize wire overlap costs. SPTA [4] proposed a track assignment algorithm based on integer linear programming and a two-stage partition strategy to provide more accurate track assignment solutions and more reliable detail routing estimation. [5] discussed how GPU parallel acceleration enhances the efficiency of each stage of VLSI design through high-performance computation and the feasibility of GPU parallel acceleration.

To obtain a better track assignment solution within a short period, this work proposes a GPU-accelerated parallel track assignment algorithm. The main contributions includes: (1) A track assignment strategy is proposed that considers both global nets and local nets, identifying challenging routed areas for detail routing. (2) A multi-strategy optimization method is proposed to explore candidates for each sub-problem of track assignment, using backtracking and reassignment to optimize the candidate solution set. (3) An asynchronous parallel acceleration strategy is proposed to fine-grain the routing resource calculation and improve the execution speed without sacrificing the solution quality. Experimental results show that, our algorithm achieves better track assignment solutions without sacrificing the quality of the track assignment.

II. PROBLEM FORMULATION

A. Problem Model

As shown in Fig. 1, in the track assignment stage, the wires generated by the global router are assigned to specific tracks. The

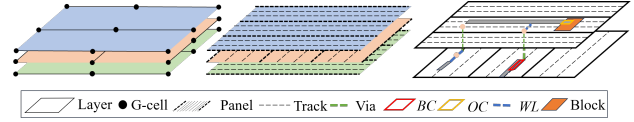


Fig. 1: Multi-layer routing model.

optimization objectives mainly include blockage cost (BC), overlap cost (OC), and wire length (WL). And the net is constructed by using the Minimum Spanning Tree (MST) method in [3].

III. DETAILS OF THE PROPOSED APPROACH

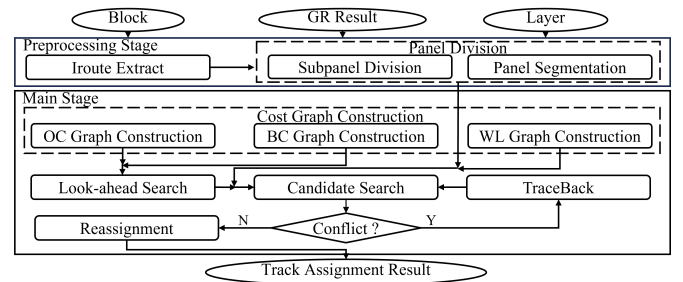


Fig. 2: The asynchronous flow of the proposed algorithm.

As shown in Fig. 2, the flow of the proposed algorithm is divided into two main stages: preprocessing and main stage.

A. Preprocessing Stage

1) Route Extract

For the global net, iroute is defined as a straight line from the center of one G-cell to the center of another G-cell. For the local nets, Steiner trees are created for each local net: vertical trunk Steiner tree and horizontal trunk Steiner tree. The trunk with a shorter wire length is selected as the wire extracted for the local net.

2) Panel Division

Before the assignment, the panels are divided into several segments based on the coordinates of each iroute. The panel is partitioned into multiple subsets called sub-panels. Iroutes in each subset do not overlap with other sub-panels on any track.

B. Main Stage of Track Assignment

1) Block Cost (BC) and Overlap Cost (OC)

The blockage cost (BC) represents the overlap length between blockages and iroutes. Overlap cost is incurred while iroutes are assigned to the same track. When considering both BC and OC, BC is essential to be prioritized, even if it necessitates compromising OC.

2) Look-ahead Search

A look-ahead search is employed to evaluate the potential conflicts. The probability evaluation values are used to facilitate the differentiation of subsequent identical sub-states. Certain iroutes are mutually exclusive due to overlapping relationships. Based on the blocks, the probabilities are divided by the tracks with the minimum bc to ensure

TABLE I: Benchmarks and Experimental Results

Case	#Block (10^4)	#Net (10^4)	#Local Net (10^4)	#Route (10^4)	NTA				SPTA				Ours			
					WL (10^6)	OC (10^6)	BC (10^6)	RT (s)	WL (10^6)	OC (10^6)	BC (10^6)	RT (s)	WL (10^6)	OC (10^6)	BC (10^6)	RT (s)
sb2	3.10	99.09	30.40	213.47	26.88	2.79	0.34	40	27.31	0.61	0.34	1144	26.39	0.61	0.34	12
sb3	1.63	89.80	33.91	201.73	25.37	1.97	0.33	32	26.03	0.80	0.33	2432	25.19	0.79	0.33	21
sb6	1.66	100.66	35.07	203.75	26.80	1.66	0.23	32	27.12	0.78	0.23	1693	26.20	0.78	0.23	12
sb7	1.11	134.04	44.04	288.32	40.38	1.56	0.21	47	40.81	0.84	0.21	1295	39.20	0.84	0.21	15
sb9	0.96	83.38	31.79	167.10	22.67	0.98	0.15	29	22.78	0.52	0.15	1032	22.01	0.52	0.15	10
sb11	1.27	93.57	27.54	181.18	23.78	0.81	0.22	25	24.52	0.45	0.22	693	23.15	0.45	0.22	12
sb12	0.37	129.34	36.19	271.97	39.12	0.82	0.04	50	39.17	0.39	0.04	1576	37.87	0.39	0.04	15
sb14	1.03	61.98	17.81	126.16	16.27	0.84	0.26	16	16.87	0.45	0.26	459	15.91	0.45	0.26	7
sb16	0.89	69.75	22.79	137.18	17.67	1.26	0.14	13	18.11	0.57	0.14	693	17.37	0.57	0.14	13
sb19	0.97	51.17	18.77	99.56	13.34	0.51	0.09	27	13.42	0.34	0.09	296	12.96	0.34	0.09	6
Ratio	-	-	-	-	1.02	2.24	1.00	2.69	1.04	1.00	1.00	88.11	1.00	1.00	1.00	1.00

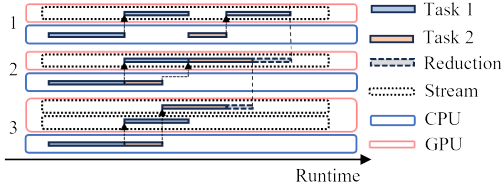


Fig. 3: Example of asynchronous strategy.

the minimization of BC . The probabilities are dynamically adjusted based on the overlap relationships, followed by the difference and summation of weights.

3) Candidates Search

Sub-state set is created for each route-to-track mapping. Sub-state includes the total cost, the index of the iroute-to-track mapping, the parent sub-state index and the boundary coordinates set of each track. If the previous candidate sub-states have the same effect on the subsequent assignment, the one with the highest probability will be chosen. When conflicts occur, tracebacking optimizes the cost further.

4) Reassignment

Some non-critical iroutes may be assigned to a track with higher costs, leading to an increase in the total cost. To reduce the occurrence of this case, a reassignment strategy is adopted. If the total cost decreases after an iroute is reassigned to another track, the iroute is considered exchangeable.

C. GPU Parallel

In heterogeneous parallel computing, the track assignment task is represented as a task dependency graph, where each computation task is a task node linked to its predecessor and successor nodes. Fig. 3 shows that multiple asynchronous scheduling enables concurrent execution of independent tasks on the CPU and GPU, reducing idle times and improving efficiency compared to sequential execution.

IV. EXPERIMENT

Our proposed algorithm was implemented using C++ and CUDA [6] with a 2.20 GHz CPU, 64 GB of memory, and an NVIDIA GeForce RTX 2080Ti. The algorithm's asynchronous threading is implemented through TaskFlow [7]. The testcases are generated initially by using NTUplace4 [8] to place the test circuits [9], followed by routing the resulting layouts with the global router NCTU-GR [8]. The routing result serves as the benchmark test circuits for the proposed algorithm. The scale of the testcases after extracting the wires from the global routing results is shown in Table I.

Table I gives the comparison of the proposed algorithm with SPTA [4] and NTA [3]. The experimental results show that our proposed algorithm achieves a speedup of 88.11 times compared to SPTA and an average speedup of 2.69 times compared to NTA. As shown in Table I, compared to the NTA algorithm, the algorithm achieves an average improvement of 51.54% in overlap cost while maintaining a

minimal blockage cost and an average improvement of 2.32% in wire length cost. In comparison with the SPTA algorithm, the proposed algorithm results in an increase of 0.25% in overlap cost and 3.94% in routing length cost on average. Compared to SPTA and NTA, our algorithm achieves better track assignment solutions with shorter execution time.

V. CONCLUSIONS

For the track assignment problem, we propose a GPU-accelerated track assignment algorithm to quickly obtain more accurate congestion prediction results. The proposed algorithm considers both global and local nets for initial assignment and incorporates optimization strategies like look-ahead search, backtracking, and reassignment to refine the candidate solutions. Furthermore, we propose an asynchronous parallel acceleration strategy to granularize the computation of routing resources without sacrificing solution quality. Experimental results show that our algorithm generates better solutions with more reliable routability results and shorter execution time.

ACKNOWLEDGMENT

This work was supported in part by the National Natural Science Foundation of China under Grant 62372109, and in part by the Fujian Natural Science Funds under Grants 2023J06017 and 2024J01984. Xing Huang is the corresponding author.

REFERENCES

- [1] L.-T. Wang, Y.-W. Chang, and K.-T. T. Cheng, *Electronic design automation: synthesis, verification, and test*. Morgan Kaufmann, 2009.
- [2] S. Batterywala, N. Shenoy, W. Nicholls, and H. Zhou, "Track assignment: A desirable intermediate step between global routing and detailed routing," in *International Conference on Computer-Aided Design*. IEEE, 2002, pp. 59–66.
- [3] M.-P. Wong, W.-H. Liu, and T.-C. Wang, "Negotiation-based track assignment considering local nets," in *Asia and South Pacific Design Automation Conference*. IEEE, 2016, pp. 378–383.
- [4] Y. Jing, L. Yang, Z. Zhuang, G. Liu, X. Huang, W.-H. Liu, and T.-C. Wang, "Spta: A scalable parallel ilp-based track assignment algorithm with two-stage partition," in *International Conference on Very Large Scale Integration*. IEEE, 2022, pp. 1–6.
- [5] Y. Lin, "Gpu acceleration in vlsi back-end design: Overview and case studies," in *International Conference on Computer-Aided Design*. IEEE, 2020, pp. 1–4.
- [6] D. Guide, "Cuda c++ programming guide," *NVIDIA*, July, 2020.
- [7] T.-W. Huang, D.-L. Lin, C.-X. Lin, and Y. Lin, "Taskflow: A lightweight parallel and heterogeneous task graph computing system," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 6, pp. 1303–1320, 2021.
- [8] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, "Nctu-gr 2.0: Multi-threaded collision-aware global routing with bounded-length maze routing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 5, pp. 709–722, 2013.
- [9] N. Viswanathan, C. Alpert, C. Sze, Z. Li, and Y. Wei, "The dac 2012 routability-driven placement contest and benchmark suite," in *Design Automation Conference*. IEEE, 2012, pp. 774–782.